

Short game values are countable

Disproof of Conjecture 00000008873

ziangni-sys 4 October 2026

Claim and actual game class

Both originals define short games as the inductive class of finite games and assert that its cardinality is the continuum, realized by binary game trees. This cardinal assertion is false. We prove that the set of actual short combinatorial game values has cardinality at most $\aleph_0$, strictly smaller than $\mathfrak{c} = 2^{\aleph_0}$.

A finite game form consists of two finite lists of previously constructed game forms: its Left options and Right options. Any finite option set can be listed, so these descriptions allow arbitrary finite branching. They are not restricted to one move for each player. Listing options may introduce order or redundant descriptions; passing to actual game values identifies these redundancies and cannot increase cardinality.

Finite descriptions represent every short game

Use finite syntax with three constructors:

$$c ::= \text{nil} \mid \text{cons}(c, c) \mid \text{node}(c, c).$$

The first two encode finite lists; the last encodes a game with two option lists. Interpret a node by the actual game construction $\{L \mid R\}$, using its decoded Left and Right option lists. For convenience, a non-node used as a game decodes to zero, and a non-cons used as an option list decodes to the empty list. All interpretations are short, by induction on the finite syntax.

Conversely, let x be any actual short pre-game. Its Left and Right move types are finite, and each move leads to another short game. By induction on shortness, choose finite descriptions of all successor games. Enumerate each finite move type by $\{0, \dots, m-1\}$, collect the chosen descriptions in those finite lists, and form a node with these two lists. Its interpretation is a *relabeling* of x : the finite enumerations give bijections of move types, and the induction hypotheses give relabelings of every corresponding successor. Relabeling implies genuine game equivalence. Thus every short game value, including all possible option arrangements, is represented. This is a proved representation result, not an assumed surjectivity condition.

Cardinal obstruction

Finite syntax is countable: encode constructor tags and the two child codes by a natural-number pairing function, recursively. Injectivity follows by recovering the tag and child codes. The interpretation map therefore gives a surjection from a countable type onto all actual short game values. A surjective image of a countable set is countable, so

$$|\text{ShortValues}| \leq \aleph_0 < 2^{\aleph_0} = \mathfrak{c}.$$

The strict inequality is Cantor's theorem. Hence the continuum-cardinality clause already fails, regardless of the subsequent leaf-depth assertion. Infinite binary paths can have continuum cardinality, but they are not finite games in the source's inductive class.

Formal correspondence and completeness

The Lean project uses Mathlib’s actual `PGame`, its inductive `PGame.Short` evidence, `PGame.Relabelling`, and the actual `Game` quotient by game equivalence. It does not replace all short games by an arbitrarily chosen countable subfamily.

The type `Code` is the finite syntax above. Mathlib’s `deriving Countable` handler produces a natural-number encoding and a kernel-checked injectivity proof. The mutually recursive `decode` and `decodeOptions` interpret codes as actual pre-games and lists of pre-games. Nodes use `PGame.ofLists`, whose move types are finite indices of the decoded option lists. The theorem `decoded_short_and_options` proves that every interpretation is short and every decoded option list consists of short games.

Crucially, `represents_every_short` quantifies over every actual short `PGame`, including arbitrary finite move-index types. By induction on the actual `Short` evidence, it enumerates those types with `Fintype.equivFin`, uses the recursively represented successors, and constructs an actual `Relabelling`. Each move bijection and each successor correspondence is proved. This removes irrelevant raw index-type labels without assuming that raw `PGame` equality has countable many encodings.

The predicate `IsShortValue v` says that some actual short pre-game represents the actual game value v . The type `ShortValue` is the subtype of all such values in the Mathlib game quotient. The map `value` sends a code to its interpretation’s actual game value, together with proved shortness. The theorem `value_surjective` proves that every member of this full subtype is reached, using the representation theorem and `Relabelling.equiv`. Its equality is equality in the actual `Game` quotient, not a new artificial relation on syntax.

Finally, `short_values_countable` follows from this actual surjection; `short_value_cardinal_lt_continuum` proves the strict cardinal inequality; and `conjecture_8873_false` proves that the claimed cardinal equality cannot hold.

Verification and scope

The proof allows every finite Left/Right option family; no bound on branching, game depth, or size is imposed. It covers short game values under standard Conway equivalence. Finite game forms modulo relabeling are likewise represented by the same syntax, so distinguishing more forms than their values does not create continuum many finite games.

Lean 4.19.0 and Mathlib commit `c44e0c8ee63ca166450922a373c7409c5d26b00b` are pinned in the project. Axiom audits of the syntax encoding, representation theorem, actual value surjection, and final negation use only `propext`, `Classical.choice`, and `Quot.sound`. No extra axioms, admitted proofs, or native evaluation shortcuts are used. No auxiliary computation is required.